

INFORME TÉCNICO - SISTEMA DE CRÉDITOS IPSFA

Versión del Sistema: v4 (Julio 2026)

Fecha de Análisis: 2026-07-14

1. ARQUITECTURA GENERAL

1.1 Tecnologías Utilizadas

Capa	Tecnología	Versión
Backend	Node.js + Express	^4.x
Base de Datos	PostgreSQL	14+
Driver DB	node-pg (Pool)	^8.x
Frontend	HTML5 + CSS3 + Vanilla JS	—
Charts	ApexCharts (CDN)	3.45.1
PDF/Excel	jsPDF + xlsx (CDN)	2.5.1 / 0.18.5
Icons	Font Awesome (CDN)	6.4.0
Auth	JWT (implícito en token localStorage)	—

1.2 Estructura de Carpetas (Esperada)

```
proyecto/
├─ server.js           # Servidor principal Express
├─ .env               # Variables de entorno
├─ package.json
├─ public/            # Archivos estáticos
│  ├─ css/
│  │  ├─ panel.css     # Estilos del panel principal
│  │  └─ tienda-caracas.css # Estilos tienda Caracas
│  └─ js/
│     ├─ panel.js      # Lógica principal del panel (SPA)
│     ├─ tienda-caracas.js # Módulo standalone Caracas
│     ├─ tienda-caracas-spa.js # Módulo SPA Caracas
│     ├─ tienda-maracay-spa.js # Módulo SPA Maracay
│     └─ reportes-caracas.js # (referenciado, no subido)
├─ index.html / login.html # Página de login
├─ panel.html          # Panel principal (SPA)
├─ tienda-caracas.html  # Vista standalone Caracas
├─ reportes-caracas.html # Reportes standalone
├─ routes/             # Archivos de rutas Express
│  ├─ auth.js          # Autenticación
│  ├─ bcv.js           # Tasas BCV
│  ├─ usuarios.js      # Gestión de usuarios
│  ├─ actividades.js   # Agenda/Actividades
│  ├─ estadisticas.js  # Estadísticas
│  └─ reportes.js      # Reportes
```

2. BASE DE DATOS - ESTRUCTURA DE TABLAS

2.1 Tabla: **tienda_caracas** (y **tienda_maracay** - misma estructura)

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	numero	INTEGER	Número correlativo
3	nro_factura	VARCHAR	Número de factura
4	nombre_apellido	VARCHAR	Nombre del cliente
5	monto_factura	DECIMAL	Monto total factura (Bs)
6	fecha_factura	DATE	Fecha de la factura
7	cedula	VARCHAR	Cédula del cliente
8	telefono	VARCHAR	Teléfono del cliente
9	monto_facturado_divisa	DECIMAL	Monto en divisa
10	dolar_facturado	DECIMAL	Monto en dólares
11	cuotas	INTEGER	Número de cuotas
12	monto_pendiente	DECIMAL	Monto pendiente
13	monto_depositados	DECIMAL	Total depositado (calculado)
14	deuda	DECIMAL	Deuda restante (calculado)
15-25	cuota_1 a cuota_11	DECIMAL	Monto de cada cuota
26-36	ref_cuota_1 a ref_cuota_11	VARCHAR	Referencia bancaria
37-47	fecha_cuota_1 a fecha_cuota_11	DATE	Fecha de depósito
48-58	tasa_cuota_1 a tasa_cuota_11	DECIMAL	Tasa BCV del día
59-69	dolar_depositado_cuota_1 a dolar_depositado_cuota_11	DECIMAL	Equivalente en USD
70	updated_at	TIMESTAMP	Última actualización

Nota: Tienda Maracay usa tabla **tienda_maracay** con **exactamente la misma estructura**.

2.2 Tabla: **usuarios**

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	nombre	VARCHAR	Nombre completo
3	email	VARCHAR	Correo electrónico
4	password	VARCHAR	Hash de contraseña
5	rol	VARCHAR	administrador o operador

Table 3 – continued

#	Campo	Tipo	Descripción
6	activo	BOOLEAN	Estado activo/inactivo
7	ip_asignada	VARCHAR	IP restringida (operadores)
8	tienda	VARCHAR	Tienda asignada (operadores)
9	fecha_creacion	TIMESTAMP	Fecha de creación
10	fecha_actualizacion	TIMESTAMP	Última actualización

2.3 Tabla: actividades

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	descripcion	TEXT	Descripción de la actividad
3	descripcion_extra	TEXT	Notas adicionales
4	hora	TIME	Hora programada
5	prioridad	VARCHAR	baja, media, alta, urgente
6	tienda	VARCHAR	caracas, maracay, maracaibo
7	fecha	DATE	Fecha de la actividad
8	estado	VARCHAR	pendiente o completada
9	fecha_completada	TIMESTAMP	Cuándo se completó
10	creado_por	INTEGER	ID del usuario creador
11	created_at	TIMESTAMP	Fecha de creación

2.4 Tabla: auditoria_usuarios

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	usuario_id	INTEGER	Usuario afectado
3	usuario_accion_id	INTEGER	Usuario que realizó la acción
4	accion	VARCHAR	Tipo de acción
5	datos_anteriores	JSONB	Datos antes del cambio
6	datos_nuevos	JSONB	Datos después del cambio
7	ip_address	VARCHAR	IP del usuario
8	created_at	TIMESTAMP	Fecha del evento

2.5 Tabla: bcv_tasas (inferida)

#	Campo	Tipo	Descripción
1	id	SERIAL PK	ID autoincremental
2	fecha	DATE	Fecha de la tasa
3	usd	DECIMAL	Tasa USD/VES
4	eur	DECIMAL	Tasa EUR/VES
5	created_at	TIMESTAMP	Fecha de registro

3. API REST - ENDPOINTS COMPLETOS

3.1 Módulo: Autenticación (/api/auth)

Método	Endpoint	Descripción	Archivo
POST	/api/auth/login	Login con email/ password	auth.js

Request:

```
{
  "email": "usuario@ejemplo.com",
  "password": "contraseña123"
}
```

Response:

```
{
  "exito": true,
  "token": "jwt_token_aqui",
  "usuario": {
    "id": 1,
    "nombre": "Admin",
    "email": "admin@ipsfa.com",
    "rol": "administrador",
    "tienda": "caracas"
  }
}
```

3.2 Módulo: BCV (/api/bcv)

Método	Endpoint	Descripción	Archivo
GET	/api/bcv/actual	Tasa actual	bcv.js
GET	/api/bcv/anterior	Tasa anterior	bcv.js
GET	/api/bcv/fechas	Listado de fechas disponibles	bcv.js
GET	/api/bcv/fecha/:fecha	Tasa por fecha específica	bcv.js + server.js (fallback)

Table 8 – continued

Método	Endpoint	Descripción	Archivo
GET	/api/bcv/ historial/:year	Historial anual	bcv.js

Response típico:

```
{
  "exito": true,
  "tasa": {
    "current": { "usd": 721.3456, "eur": 785.4321, "date": "2026-07-14" },
    "previous": { "usd": 720.1234, "eur": 784.5678, "date": "2026-07-13" },
    "changePercentage": { "usd": 0.17, "eur": 0.11 }
  }
}
```

3.3 Módulo: Usuarios (/api/usuarios)

Método	Endpoint	Descripción	Archivo
GET	/api/usuarios	Listar usuarios (con filtros)	usuarios.js
GET	/api/usuarios/:id	Obtener usuario por ID	usuarios.js
POST	/api/usuarios	Crear usuario	usuarios.js
PUT	/api/usuarios/:id	Actualizar usuario	usuarios.js
DELETE	/api/usuarios/:id	Desactivar usuario	usuarios.js
PATCH	/api/usuarios/:id/ reactivar	Reactivar usuario	usuarios.js
PUT	/api/usuarios/:id/ password	Cambiar contraseña	usuarios.js
POST	/api/usuarios/ recuperar-password	Solicitar recuperación	usuarios.js
POST	/api/usuarios/ restablecer-password	Restablecer con token	usuarios.js
GET	/api/usuarios/perfil/me	Perfil del usuario logueado	usuarios.js
GET	/api/usuarios/ auditoria/:usuarioId	Historial de auditoría	usuarios.js
GET	/api/usuarios/ estadisticas/resumen	Estadísticas de usuarios	usuarios.js

Query params para GET /api/usuarios:

- busqueda - Filtrar por nombre/email
- rol - administrador o operador
- activo - true o false

3.4 Módulo: Tienda Caracas (/api/tienda-caracas) - INLINE en server.js

Método	Endpoint	Descripción
GET	/api/tienda-caracas	Listar todos los clientes
GET	/api/tienda-caracas/:id	Obtener cliente por ID
POST	/api/tienda-caracas	Crear nuevo cliente + cuota 1
PUT	/api/tienda-caracas/:id	Actualizar cliente (cuotas, teléfono)
DELETE	/api/tienda-caracas/:id	Eliminar cliente

3.5 Módulo: Tienda Maracay (/api/tienda-maracay) - INLINE en server.js

Método	Endpoint	Descripción
GET	/api/tienda-maracay	Listar todos los clientes
GET	/api/tienda-maracay/:id	Obtener cliente por ID
POST	/api/tienda-maracay	Crear nuevo cliente + cuota 1
PUT	/api/tienda-maracay/:id	Actualizar cliente (cuotas, teléfono)
DELETE	/api/tienda-maracay/:id	Eliminar cliente

3.6 Módulo: Actividades (/api/actividades)

Método	Endpoint	Descripción	Archivo
GET	/api/actividades	Listar actividades	actividades.js
GET	/api/actividades/:id	Obtener actividad por ID	actividades.js
POST	/api/actividades	Crear actividad	actividades.js
PUT	/api/actividades/:id	Actualizar actividad	actividades.js
DELETE	/api/actividades/:id	Eliminar actividad	actividades.js
GET	/api/actividades/ estadisticas/resumen	Resumen estadístico	actividades.js

Query params para GET:

- fecha - Filtrar por fecha (YYYY-MM-DD)
- tienda - Filtrar por tienda (caracas, maracay, maracaibo)
- estado - pendiente o completada
- fecha_desde / fecha_hasta - Rango de fechas

3.7 Módulo: Estadísticas (/api/estadisticas)

Método	Endpoint	Descripción	Archivo
GET	/api/estadisticas	Resumen general	estadisticas.js
GET	/api/estadisticas/ deudores	Listado de deudores	estadisticas.js

Table 13 – continued

Método	Endpoint	Descripción	Archivo
GET	/api/estadisticas/evolucion	Evolución mensual	estadisticas.js
GET	/api/estadisticas/distribucion	Distribución de pagos	estadisticas.js
GET	/api/estadisticas/exportar	Exportar datos	estadisticas.js

3.8 Módulo: Reportes (/api/reportes)

Método	Endpoint	Descripción	Archivo
POST	/api/reportes/caracas	Generar reporte filtrado	reportes.js

Body del POST:

```
{
  "fecha_desde": "2026-01-01",
  "fecha_hasta": "2026-07-14",
  "estado": "todos",
  "monto_min": 0,
  "monto_max": 9999999,
  "nombre_cliente": ""
}
```

4. FRONTEND - ESTRUCTURA DE ARCHIVOS

4.1 Archivo: panel.html (SPA Principal)

Rol: Página principal del sistema. Single Page Application (SPA) que carga todos los módulos.

Secciones (content-area):

ID	Nombre	Descripción
contentDashboard	Panel Principal	Reloj, tasa BCV, widget actividades, placeholder
contentTasas	Tasas BCV	Consulta por fecha + historial
contentUsuarios	Gestión de Usuarios	CRUD de usuarios, estadísticas, filtros
contentPerfil	Mi Perfil	Datos personales, cambio de contraseña
contentClientes	Tienda Caracas	Módulo completo Caracas (menú → base datos → conciliaciones → reportes)

Table 15 – continued

ID	Nombre	Descripción
contentCreditos	Tienda Maracay	Módulo completo Maracay (misma estructura que Caracas)
contentPagos	Tienda Maracaibo	Placeholder - “En desarrollo”
contentReportes	Actividades Pendientes	Agenda digital con CRUD de actividades
contentEstadisticas	Estadísticas	KPIs, gráficos ApexCharts, tabla de deudores

Navegación Sidebar (**nav-item**):

- nav-dashboard → Panel Principal
- nav-caracas → Tienda Caracas
- nav-maracay → Tienda Maracay
- nav-maracaibo → Tienda Maracaibo
- nav-tasas → Tasas BCV
- nav-usuarios → Usuarios

Modales:

- modalUsuario / modalUsuarioTitulo - Crear/Editar usuario
- modalConfirmar - Confirmación de acciones
- modalAuditoria - Historial de auditoría
- modalFecha / modalFechaInput / modalResultado - Consulta tasa por fecha
- modal-corporativo - Modal genérico reutilizable (icono, título, mensaje, botones)

Scripts cargados:

```
<script src="js/panel.js"></script>
<script src="js/tienda-caracas-spa.js"></script>
<script src="js/tienda-maracay-spa.js"></script>
<script src="https://cdn.jsdelivr.net/npm/apexcharts@3.45.1/dist/apexcharts.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf-autotable/3.5.31/
  ↳ jspdf.plugin.autotable.min.js"></script>
```

4.2 Archivo: **panel.js** (Lógica Principal SPA)

Tamaño: ~3,500 líneas | **Funciones:** 94 declaradas

Variables Globales:

```
const token = localStorage.getItem('token');
let usuario = {}; // Datos del usuario logueado
let usuariosData = []; // Cache de usuarios
let usuarioEditando = null; // ID en edición
```



```

let chartEvolucion = null;           // Instancia ApexCharts
let chartDistribucion = null;        // Instancia ApexCharts
let accionPendiente = null;          // Para modal de confirmación
let usuarioPendiente = null;         // Para modal de confirmación
let datosEstadisticasCache = null;   // Cache de estadísticas
let actividades = [];                // Cache de actividades
let filtroActividades = 'pendientes'; // Filtro actual agenda
let paginaHistorial = 1;              // Paginación historial
let totalPaginasHistorial = 1;        // Total páginas historial
let actividadesHistorialCache = [];
let datosBusquedaCaracas = [];       // Cache reportes Caracas
let resumenBusquedaCaracas = {};      // Resumen reportes
let paginaBusqueda = 1;               // Paginación reportes
let registrosPorPaginaBusqueda = 10;
let totalPaginasBusqueda = 1;

```

Secciones principales del código:

Sección	Funciones Clave	Descripción
Autenticación	sincronizarPerfilUsuario(), cerrarSesion()	Login, logout, verificación de token
Sidebar	toggleSidebar(), mostrarSeccion()	Navegación entre secciones, protección por rol
Reloj	actualizarReloj()	Reloj en tiempo real, calendario semanal
Tasa BCV	cargarTasaActual(), consultarTasaFecha(), cargarHistorial()	Consulta de tasas, historial
Usuarios	cargarUsuarios(), renderizarUsuarios(), guardarUsuario(), editarUsuario(), confirmarEliminar(), confirmarReactivar()	CRUD completo de usuarios
Perfil	cargarPerfil(), formCambiarPassword	Datos personales, cambio de password
Estadísticas	initEstadisticas(), cargarDatosEstadisticasReales(), procesarDatosEstadisticas(), actualizarKPIsReales(), actualizarGraficosReales()	Dashboard con ApexCharts
Agenda	initAgenda(), agregarActividad(), toggleActividad(), eliminarActividad(), editarActividad(), cargarHistorialActividades()	CRUD de actividades del día
Widget Actividades	actualizarWidgetActividades(), completarActividadDesdeWidget(), abrirModalDescripcionWidget()	Widget en dashboard

Table 16 – continued

Sección	Funciones Clave	Descripción
Reportes Caracas	<code>mostrarBusqueda()</code> , <code>generarBusquedaCaracas()</code> , <code>renderizarTablaBusqueda()</code> , <code>exportarBusquedaExcel()</code> , <code>exportarBusquedaPDF()</code>	Reportes con filtros, paginación, exportación
Responsive	<code>esMovil()</code> , <code>ajustarVistaMovil()</code>	Detección móvil, adaptación de UI
Control de Acceso	<code>isAdmin()</code> , <code>getUserRole()</code> , <code>getTiendaUsuario()</code> , <code>ocultarMenuSegunRol()</code>	Roles: admin (todo) vs operador (solo su tienda)

Flujo de navegación:

1. Usuario hace clic en nav-item → `mostrarSeccion('clientes')`
2. Se ocultan todas las `content-area` y se muestra la seleccionada
3. Se verifica permiso según rol (`isAdmin()` / `getTiendaUsuario()`)
4. Se ejecuta función de inicialización específica (ej: `initTiendaCaracas()`)
5. Se cargan datos desde API

4.3 Archivo: `tienda-caracas-spa.js` (Módulo SPA Caracas)

Tamaño: ~1,500 líneas | **Funciones:** 61 | **Scope:** Dentro de `panel.html` (sección `contentClientes`)

Variables Globales (dentro del script):

```
const API_BASE_URL = window.location.origin + '/api';
const ITEMS_PER_PAGE_DEFAULT = 25;
let allData = [];           // Todos los clientes cargados
let filteredData = [];      // Datos filtrados
let currentPage = 1;
let itemsPerPage = 25;
let currentFilter = 'all';
let isInitialized = false;
let currentEditId = null;   // Cliente en edición
let currentEditItem = null; // Datos del cliente en edición
let conciliacionClienteActual = null; // Para conciliaciones
let conciliacionTasaActual = null;    // Tasa BCV actual
let cuotasAEliminar = [];   // Cuotas marcadas para eliminar
```

Sub-módulos dentro de Caracas:

Sub-módulo	Funciones Clave	Descripción
Base de Datos	<code>initTiendaCaracas()</code> , <code>loadData()</code> , <code>renderTable()</code> , <code>applyFilters()</code> , <code>updateSummary()</code>	Tabla paginada con filtros rápidos y avanzados

Table 17 – continued

Sub-módulo	Funciones Clave	Descripción
Modal Edición	<code>verDetalle()</code> , <code>createModalElement()</code> , <code>fillFormData()</code> , <code>guardarCambios()</code> , <code>closeModal()</code>	Ver/editar cliente, cuotas, teléfono. Admin=edita, Operador=solo lee
Eliminar Cuotas	<code>confirmarEliminarCuotas()</code> , <code>ejecutarEliminarCuotas()</code>	Checklist de cuotas a eliminar, recálculo de deuda
Conciliaciones	<code>mostrarConciliaciones()</code> , <code>buscarFacturaConciliacion()</code> , <code>mostrarClienteEncontrado()</code> , <code>guardarCuotaConciliacion()</code>	Buscar por factura, agregar cuotas, verificar deuda=0
Nuevo Registro	<code>mostrarFormularioNuevoRegistro()</code> , <code>guardarNuevaConciliacion()</code>	Crear cliente + cuota 1
Tasa BCV	<code>obtenerTasaPorFechaConciliacion()</code> , <code>obtenerTasaNuevaConciliacion()</code>	Consulta tasa por fecha, fallback a actual, fallback hardcodeado
Exportación	<code>exportToExcel()</code> , <code>exportToPDF()</code> , <code>printTable()</code>	CSV, PDF con jsPDF, impresión
Modal Corporativo	<code>mostrarModalCorporativo()</code> , <code>cerrarModalCorporativo()</code>	Modal reutilizable con icono, título, mensaje, botones personalizados
Eliminar Cliente	<code>confirmarEliminarCliente()</code> , <code>eliminarCliente()</code>	Confirmación + eliminación API

Flujo de Base de Datos:

1. `initTiendaCaracas()` → carga datos desde `/api/tienda-caracas`
2. `processItemData()` → calcula: `monto_depositados` (suma cuotas), `deuda` (`monto_factura` - `depositado`)
3. `renderTable()` → muestra tabla paginada con: N°, Factura, Cliente, Monto, Fecha, Cédula, Cuotas, Depositado, Deuda, Estado, Acciones
4. `applyFilters()` → filtra por: búsqueda general, factura, cédula, fechas, montos, estado (todos/deudores/incompletos/al día/abiertas/canceladas)

Flujo de Conciliaciones:

1. Usuario ingresa N° de factura → `buscarFacturaConciliacion()`
2. Si existe: muestra info + historial de cuotas + formulario para nueva cuota
3. Si `deuda=0`: modal “Factura Cancelada” → opción de agregar cuota adicional
4. Si no existe: modal “Factura No Encontrada” → opción de crear nuevo registro
5. Formulario cuota: monto Bs → consulta tasa BCV → calcula dólar depositado → guarda

Reglas de Negocio Críticas:

- `deuda = monto_factura - monto_depositados`

- `monto_depositados = suma(cuota_1 ... cuota_11)` donde `cuota_X > 0`
- Si `deuda <= 0` → estado = “cancelada” (factura pagada)
- Si `deuda > 0` y `cuotas_pagadas = 0` → estado = “abierta”
- Si `deuda > 0` y `cuotas_pagadas > 0` → estado = “incompleto”
- Solo admin puede editar cuotas. Operador solo ve (teléfono editable para ambos)

4.4 Archivo: **tienda-maracay-spa.js** (Módulo SPA Maracay)

Tamaño: ~1,500 líneas | **Funciones:** 61 | **Scope:** Dentro de `panel.html` (sección `contentCredits`)

Estructura IDÉNTICA a `tienda-caracas-spa.js` pero:

- Variables terminan en `M` (ej: `allDataM`, `currentPageM`, `initTiendaMaracay()`)
- API endpoint: `/api/tienda-maracay` en lugar de `/api/tienda-caracas`
- Envuelto en IIFE (`function() { ... }()`) para aislar variables
- Mismo flujo, mismas funciones, mismas reglas de negocio

4.5 Archivo: **tienda-caracas.js** (Módulo Standalone)

Tamaño: ~600 líneas | **Funciones:** 30 | **Scope:** `tienda-caracas.html` (página independiente)

Diferencias con la versión SPA:

- No tiene menú principal ni conciliaciones
- Tiene sidebar propio con navegación básica
- Funciones más simples (sin modal de edición completo)
- Usa `API_BASE_URL = 'http://localhost:3000/api'` (hardcodeado)
- Tiene datos de ejemplo (`getSampleData()`) como fallback
- **NO está integrado en el panel principal** - es una vista alternativa

4.6 Archivos CSS

`panel.css` (~1,800 líneas):

- Variables CSS: `--primary: #1a365d`, `--secondary: #667eea`, `--success: #48bb78`, etc.
- Layout: Sidebar fijo (240px) + Main content flexible
- Componentes: Cards, tablas, formularios, modales, badges, KPIs, gráficos
- Responsive: Breakpoints en 1440px, 1200px, 1024px, 768px, 480px
- Sidebar colapsa a 70px en desktop, se oculta en móvil (hamburguesa)
- Estilos específicos para: estadísticas, tienda caracas, agenda, widget actividades

`tienda-caracas.css` (~800 líneas):

- Variables: `--primary: #1a3a5c`, `--success: #28a745`, etc.
- Componentes similares a `panel.css` pero más enfocado

- Responsive para tabla con scroll horizontal
 - Estilos para modal de edición, formularios, cuotas
-

5. ROLES Y PERMISOS

5.1 Administrador (rol: 'administrador')

- ☒ Acceso a TODAS las secciones
- ☒ CRUD completo de usuarios
- ☒ Editar cuotas de clientes
- ☒ Eliminar cuotas
- ☒ Ver auditoría
- ☒ Ver actividades de TODAS las tiendas
- ☒ Estadísticas globales

5.2 Operador (rol: 'operador')

- ☒ Solo su tienda asignada (tienda: 'caracas', 'maracay' o 'maracaibo')
- ☒ Ver clientes de su tienda (solo lectura, excepto teléfono)
- ☒ Agregar cuotas (conciliaciones)
- ☒ Crear nuevos clientes
- ☒ Ver y crear actividades de su tienda
- ☒ NO puede ver usuarios
- ☒ NO puede ver actividades de otras tiendas
- ☒ NO puede editar cuotas existentes
- ☒ NO puede eliminar cuotas
- ☒ NO puede acceder a estadísticas globales

5.3 Control de Acceso en Código

```
// panel.js - mostrarSeccion()
if (!isAdmin()) {
  const tiendaUsuario = getTiendaUsuario();
  // Actividades: solo admin
  if (seccion === 'reportes') { mostrarAlerta('No tienes permiso...'); return; }
  // Tiendas: solo la asignada
  const tiendaMap = { 'clientes': 'caracas', 'creditos': 'maracay', 'pagos': 'maracaibo' };
  if (tiendaMap[seccion] !== tiendaUsuario) { mostrarAlerta('No tienes permiso...'); return; }
  // Usuarios: solo admin
  if (seccion === 'usuarios') { mostrarAlerta('No tienes permiso...'); return; }
}
```

6. FLUJOS DE TRABAJO CLAVE

6.1 Flujo: Crear Cliente + Primera Cuota (Conciliaciones)

```
1. Usuario va a Tienda Caracas → Conciliaciones Bancarias
```

2. Ingresar N° de factura
3. Si NO existe → "Factura No Encontrada" → "Registrar Nuevo Cliente"
4. Formulario:
 - N° Factura (readonly, del input anterior)
 - Fecha Factura
 - Nombre y Apellido
 - Cédula
 - Monto Factura (Bs)
 - Primera Cuota:
 - * Monto (Bs) → oninput: calcularDolarNuevaConciliacion()
 - * Referencia
 - * Fecha Depósito → onchange: obtenerTasaNuevaConciliacion()
 - * Tasa BCV → calculada automáticamente
 - * Dólar Depositado → calculado (monto / tasa)
5. Guardar → POST /api/tienda-caracas
 Body: { nro_factura, nombre_apellido, cedula, monto_factura, fecha_factura, monto_depositados, deuda, cuota_1, ref_cuota_1, fecha_cuota_1, tasa_cuota_1, dolar_depositado_cuota_1 }

6.2 Flujo: Agregar Cuota a Cliente Existente

1. Usuario va a Conciliaciones → ingresa N° factura
2. Si existe → muestra info del cliente + historial de cuotas
3. Formulario nueva cuota:
 - N° Cuota (auto: cuotas_pagadas + 1)
 - Monto (Bs)
 - Referencia
 - Fecha Depósito → consulta tasa BCV
 - Tasa BCV
 - Dólar Depositado
4. Guardar → PUT /api/tienda-caracas/:id
 Body: { cuota_X, ref_cuota_X, fecha_cuota_X, tasa_cuota_X, dolar_depositado_cuota_X, monto_depositados, deuda }
5. Si deuda después = 0 → modal "Factura Cancelada"

6.3 Flujo: Editar Cliente (Admin)

1. Base de Datos → clic en "Ver y editar" (ojo)
2. Modal se crea DINÁMICAMENTE según rol:
 - Admin: inputs editables para cuotas
 - Operador: divs readonly (solo teléfono editable)
3. Se muestran SOLO las cuotas que existen en BD (cuota_X > 0)
4. Admin puede editar: monto, referencia, fecha, tasa de cada cuota
5. Sección "Eliminar Cuotas" con checklist (solo admin)
6. Guardar → PUT /api/tienda-caracas/:id
 - Recalcula monto_depositados y deuda
 - Preserva monto_factura original (CRÍTICO: no lo lee del input formateado)

6.4 Flujo: Reportes Caracas

1. Tienda Caracas → Reportes
2. Filtros: fecha_desde, fecha_hasta, estado, monto_min, monto_max, nombre
3. Generar → POST /api/reportes/caracas
4. Respuesta: { exito, datos[], resumen { total_clientes, total_deuda, total_depositado, clientes_mora, promedio_deuda } }
5. Muestra:
 - Cards resumen (5 KPIs)
 - Tabla paginada (10/25/50/100 registros por página)
 - Gráficos: barras (deuda por estado) + pastel (distribución)
 - Botones exportar Excel y PDF

7. REGLAS DE NEGOCIO CRÍTICAS

7.1 Cálculos Financieros

```
// processItemData() - aplicado en frontend y backend
montoDepositado = suma(cuota_1 ... cuota_11) donde cuota_X > 0
deuda = monto_factura - montoDepositado
if (abs(monto_factura - montoDepositado) < 0.01) deuda = 0

// Estados:
if (deuda <= 0) → "cancelada" (factura pagada)
if (deuda > 0 && cuotas_pagadas === 0) → "abierta"
if (deuda > 0 && cuotas_pagadas > 0 && cuotas_pagadas < total_cuotas) → "incompleto"
if (deuda > 0) → "deudor" (general)
```

7.2 Parseo de Números (Formato Español/Venezolano)

```
// parseNumberES() - CRÍTICO para no perder decimales
// Ej: "39.035,88" → 39035.88 (punto=miles, coma=decimal)
// Ej: "1234.56" → 1234.56 (formato inglés, sin miles)
function parseNumberES(value) {
  if (typeof value === 'number') return value;
  let str = String(value).trim();
  const lastComma = str.lastIndexOf(',');
  const lastDot = str.lastIndexOf('.');
  if (lastComma > lastDot) {
    // Formato español: 1.234,56 → 1234.56
    cleaned = str.replace(/\. /g, '').replace(',', '.');
  } else {
    // Formato inglés o sin separador
    cleaned = str.replace(/,/g, '');
  }
  return parseFloat(cleaned);
}
```

7.3 Formateo de Moneda

```
// formatCurrency() - muestra Bs con separador español
// Ej: 39035.88 → "39.035,88 Bs"
function formatCurrency(value) {
  const num = parseFloat(value);
  return new Intl.NumberFormat('es-VE', {
    minimumFractionDigits: num % 1 === 0 ? 0 : 2,
    maximumFractionDigits: 2
  }).format(num) + ' Bs';
}
```

8. CONFIGURACIÓN Y VARIABLES DE ENTORNO

8.1 .env (Requerido)

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=creditos
DB_USER=postgres
DB_PASSWORD=admin123
PORT=3000
```

8.2 Configuración en server.js

```
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'creditos',
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'admin123'
});
```

8.3 Content Security Policy (CSP)

```
app.use((req, res, next) => {
  res.setHeader('Content-Security-Policy',
    "default-src 'self'; " +
    "script-src 'self' 'unsafe-inline' 'unsafe-eval' https://cdn.jsdelivr.net https://\n" +
    "  ↳ cdnjs.cloudflare.com; " +
    "style-src 'self' 'unsafe-inline' https://cdnjs.cloudflare.com https://cdn.jsdelivr.net; " +
    "img-src 'self' data: blob; " +
    "font-src 'self' https://cdnjs.cloudflare.com; " +
    "connect-src 'self' http://localhost:* https://localhost:* https://cdnjs.cloudflare.com https://\n" +
    "  ↳ cdn.jsdelivr.net;");
  next();
});
```


9. PUNTOS DE ATENCIÓN Y POSIBLES PROBLEMAS

9.1 Problemas Conocidos

#	Problema	Ubicación	Impacto	Solución Temporal
1	BCV fallback hardcodeado	server.js línea ~55	Tasa fija 721.3456 si API BCV falla	Revisar bcv.js para conexión real
2	Duplicación de código	tienda-caracas- spa.js vs tienda-maracay- spa.js	Mantenimiento doble	Refactorizar a módulo genérico
3	API_BASE_URL hardcodeado	tienda-caracas.js	No funciona en producción	Cambiar a window.location.origin
4	Sin manejo de errores de red	Varios fetch	App se cuelga si backend cae	Agregar try/catch en todos los fetch
5	No hay validación de token expirado	panel.js	401 no redirige a login	Agregar interceptor de fetch
6	Tienda Maracaibo no implementada	panel.html	Placeholder vacío	Desarrollar módulo similar a Caracas/Maracay
7	Datos de ejemplo en tienda-caracas.js	tienda-caracas.js	Muestra datos falsos si API falla	Eliminar o hacer opcional
8	Falta índice en búsqueda de factura	server.js	Búsqueda O(n) en todos los registros	Agregar índice DB en nro_factura
9	No hay paginación en backend	server.js	Carga TODOS los registros	Implementar LIMIT/OFFSET
10	CSS duplicado	panel.css + tienda-caracas.css	Conflicto de estilos posible	Unificar CSS o usar scoped

9.2 Dependencias CDN (Sin package.json local)

- **ApexCharts:** <https://cdn.jsdelivr.net/npm/apexcharts@3.45.1>
- **jsPDF:** <https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1>
- **jsPDF-autotable:** <https://cdnjs.cloudflare.com/ajax/libs/jspdf-autotable/3.5.31>
- **xlsx:** <https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5> (solo en reportes-caracas.html)
- **Font Awesome:** <https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0>

10. GLOSARIO DE FUNCIONES POR ARCHIVO

10.1 panel.js - Funciones Principales (94 total)

Autenticación y Sesión:

- `sincronizarPerfilUsuario()` - Sincroniza datos del usuario desde API

- cerrarSesion() - Limpia localStorage y redirige a login

Navegación:

- mostrarSeccion(seccion) - Muestra sección, verifica permisos, ejecuta init
- toggleSidebar() - Abre/cierra sidebar (móvil/desktop)
- esMovil() - Detecta si es dispositivo móvil
- ajustarVistaMovil() - Adapta UI para móvil

Reloj:

- actualizarReloj() - Actualiza reloj, fecha, calendario semanal

Tasa BCV:

- cargarTasaActual() - Carga tasa actual desde API
- consultarTasaFecha() - Abre modal para consultar fecha
- buscarTasaModal() - Consulta tasa específica
- cargarHistorial() - Carga historial de tasas

Usuarios:

- cargarUsuarios() - GET /api/usuarios con filtros
- renderizarUsuarios() - Renderiza tabla de usuarios
- filtrarUsuarios() - Aplica filtros de búsqueda
- abrirModalUsuario() - Abre modal crear/editar
- guardarUsuario(event) - POST/PUT usuario
- editarUsuario(id) - Carga usuario para edición
- confirmarEliminar(id, nombre) - Modal confirmar desactivar
- confirmarReactivar(id, nombre) - Modal confirmar reactivar
- ejecutarAccionPendiente() - Ejecuta acción confirmada
- cargarEstadisticasUsuarios() - Carga contadores de usuarios

Perfil:

- cargarPerfil() - Carga datos del perfil logueado
- formCambiarPassword (event listener) - Cambio de contraseña

Estadísticas:

- initEstadisticas() - Inicializa gráficos ApexCharts
- cargarDatosEstadisticasReales() - Carga datos desde API
- procesarDatosEstadisticas(clientes, mes, anio, tipo) - Procesa datos crudos
- actualizarKPIsReales(kpis) - Actualiza cards de KPIs
- actualizarGraficosReales(evolucion, distribucion) - Actualiza charts

- `actualizarTablaDeudoresReales(deudores)` - Renderiza tabla deudores
- `cargarMesesDisponibles()` - Llena dropdown de meses con datos
- `inicializarGraficos()` - Crea instancias ApexCharts
- `aplicarFiltros()` - Aplica filtros y recarga datos

Agenda (Actividades):

- `initAgenda()` - Inicializa agenda del día
- `agregarActividad()` - Crea nueva actividad
- `toggleActividad(id)` - Marca/desmarca como completada
- `eliminarActividad(id)` - Elimina actividad con confirmación
- `editarActividad(id)` - Edita descripción de actividad
- `filtrarActividades(filtro)` - Filtra por todas/pendientes/completadas
- `renderizarActividades()` - Renderiza lista de actividades
- `actualizarStatsAgenda()` - Actualiza contadores (total/pendientes/completadas)
- `cargarHistorialActividades()` - Carga historial paginado
- `renderizarHistorialPaginado()` - Renderiza historial con paginación
- `cambiarPaginaHistorial(direccion)` - Navega páginas del historial

Widget Actividades (Dashboard):

- `actualizarWidgetActividades()` - Actualiza widget en dashboard
- `completarActividadDesdeWidget(id)` - Completa desde widget
- `abrirModalDescripcionWidget(id)` - Abre modal para añadir nota
- `guardarDescripcionWidget(id)` - Guarda descripción extra
- `mostrarModalTodasActividadesOperador()` - Modal con todas las actividades (operador)
- `completarActividadDesdeModalOperador(id)` - Completa desde modal operador

Reportes Caracas:

- `mostrarBusqueda()` - Muestra sección de reportes
- `generarBusquedaCaracas()` - Genera reporte con filtros
- `renderizarTablaBusqueda()` - Renderiza tabla paginada
- `actualizarControlesPaginacionBusqueda()` - Crea controles de paginación
- `irAPaginaBusqueda(pagina)` - Navega páginas
- `calcularEstadoBusqueda(row)` - Calcula estado del cliente
- `renderizarGraficosBusqueda()` - Renderiza gráficos de barras y pastel
- `exportarBusquedaExcel()` - Exporta a Excel (xlsx)
- `exportarBusquedaPDF()` - Exporta a PDF con jsPDF

- `limpiarFiltrosBusqueda()` - Limpia todos los filtros

Utilidades:

- `mostrarAlerta(mensaje, tipo)` - Muestra toast notification
- `formatearMoneda(valor)` - Formatea número a moneda
- `formatearFecha(dateString)` - Formatea fecha
- `getBadgeClass(estado)` - Devuelve clase CSS según estado
- `filtrarDeudores()` - Filtra tabla de deudores
- `verCliente(cedula)` - Navega a cliente específico
- `exportarReporte()` - Exporta CSV de estadísticas
- `ocultarMenuSegunRol()` - Oculta menús según rol
- `isAdmin()` - Verifica si es administrador
- `getUserRole()` - Obtiene rol del usuario
- `getTiendaUsuario()` - Obtiene tienda asignada

10.2 tienda-caracas-spa.js - Funciones Principales (61 total)

Inicialización:

- `initTiendaCaracas()` - Inicializa módulo Caracas
- `loadData()` - Carga datos desde API
- `processItemData(item)` - Procesa datos crudos (calcula deuda, depositado)

Base de Datos:

- `applyQuickFilter(filter)` - Aplica filtro rápido (todos/deudores/etc)
- `applyFilters()` - Aplica filtros avanzados
- `clearFilters()` - Limpia todos los filtros
- `debouncedFilter()` - Filtro con debounce (300ms)
- `updateSummary()` - Actualiza cards de resumen
- `updateFilterCounts()` - Actualiza contadores de filtros
- `renderTable()` - Renderiza tabla paginada
- `createRowHTML(item)` - Crea HTML de una fila
- `updatePagination()` - Actualiza controles de paginación
- `getTotalPages()` - Calcula total de páginas
- `goToPage(page)` - Navega a página
- `changeItemsPerPage()` - Cambia registros por página
- `getEstado(item)` - Determina estado del cliente
- `countCuotasPagadas(item)` - Cuenta cuotas pagadas

Modal Edición:

- `verDetalle(id)` - Abre modal de edición/ver
- `createModalElement()` - Crea HTML del modal dinámicamente
- `fillFormData(item)` - Llena formulario con datos del cliente
- `guardarCambios()` - Guarda cambios vía PUT
- `closeModal()` - Cierra modal
- `editarCliente(id)` - Alias de `verDetalle`

Eliminar Cuotas:

- `confirmarEliminarCuotas()` - Muestra confirmación con checklist
- `ejecutarEliminarCuotas(datosConfirmacion)` - Ejecuta eliminación vía PUT

Eliminar Cliente:

- `confirmarEliminarCliente(id)` - Modal de confirmación
- `eliminarCliente(id)` - Elimina vía DELETE

Conciliaciones:

- `mostrarConciliaciones()` - Muestra sección de conciliaciones
- `buscarFacturaConciliacion()` - Busca factura por número
- `mostrarClienteEncontrado(cliente)` - Muestra info del cliente
- `mostrarFormularioCuota(cliente)` - Muestra formulario nueva cuota
- `ocultarFormularioCuota()` - Oculta formulario
- `guardarCuotaConciliacion()` - Guarda nueva cuota vía PUT
- `volverABuscarFactura()` - Resetea a búsqueda inicial

Nuevo Registro:

- `mostrarFacturaNoEncontrada(nroFactura)` - Muestra mensaje no encontrada
- `mostrarFormularioNuevoRegistro()` - Muestra formulario nuevo cliente
- `mostrarNuevoRegistro(nroFactura)` - Prepara formulario con N° factura
- `guardarNuevaConciliacion()` - Guarda nuevo cliente vía POST

Tasa BCV:

- `obtenerTasaPorFechaConciliacion()` - Consulta tasa para cuota existente
- `obtenerTasaNuevaConciliacion()` - Consulta tasa para nueva cuota
- `calcularDolarConciliacion()` - Calcula dólar desde monto y tasa
- `calcularDolarNuevaConciliacion()` - Calcula dólar para nuevo registro

Historial:

- `cargarHistorialCuotas(cliente)` - Carga tabla de cuotas pagadas

Exportación:

- `exportToExcel()` - Exporta a CSV
- `exportToPDF()` - Muestra alerta (no implementado)
- `printTable()` - Imprime tabla
- `downloadFile()` - Descarga archivo generado

Navegación:

- `mostrarBaseDatos()` - Muestra base de datos
- `mostrarMenuPrincipal()` - Muestra menú principal

Modal Corporativo:

- `mostrarModalCorporativo(titulo, mensaje, tipo, botones)` - Modal genérico
- `cerrarModalCorporativo()` - Cierra modal genérico

Utilidades:

- `formatCurrency(value)` - Formatea moneda Bs
- `formatNumber(value)` - Formatea número con separador
- `formatDate(dateString)` - Formatea fecha
- `showLoading(show)` - Muestra/oculta loading
- `parseNumberES(value)` - Parsea número formato español

10.3 tienda-maracay-spa.js - Funciones (61 total, mismas con sufijo M)

Todas las funciones de `tienda-caracas-spa.js` pero con:

- Variables: `allDataM`, `currentPageM`, `itemsPerPageM`, etc.
- Funciones: `initTiendaMaracay()`, `loadDataM()`, `applyFiltersM()`, etc.
- API: `/api/tienda-maracay` en lugar de `/api/tienda-caracas`
- Envuelto en IIFE: `(function() { ... })()`

11. HISTORIAL DE CAMBIOS Y EVOLUCIÓN

11.1 Versiones Anteriores (según memoria del sistema)

- **v1-v3:** Sistema base con IndexedDB local, sin backend
- **v4:** Migración a PostgreSQL + Node.js backend
- **v4.1:** Agregado autenticación JWT, roles admin/operador
- **v4.2:** Agregado módulo de actividades/agenda
- **v4.3:** Agregado reportes con filtros avanzados y exportación
- **v4.4:** Agregado Tienda Maracay (estructura idéntica a Caracas)
- **v4.5:** Agregado widget de actividades en dashboard, descripciones extra
- **v4.6:** Agregado eliminación de cuotas individuales, corrección de parseo numérico
- **v4.7:** Agregado conciliaciones con verificación de factura cancelada

11.2 Bugs Corregidos Históricamente

1. **parseNumberES:** Valores como “39.035,88” se parseaban como 39.035 (perdía .88). Solución: detectar último separador.
 2. **monto_factura en edición:** Se leía del input formateado, causando pérdida de decimales. Solución: usar valor original del objeto.
 3. **Duplicado escapeHtml:** Conflicto entre app.js y auth.js. Solución: setUtils() para pasar funciones.
 4. **IndexedDB borraba datos:** onupgradeneeded eliminaba store. Solución: crear solo si no existe.
 5. **Tasa BCV CDN:** Fallo de carga rompía app. Solución: fallback hardcodeado + try/catch aislado.
-

12. CHECKLIST PARA NUEVOS DESARROLLADORES

12.1 Para Agregar una Nueva Tienda (ej: Maracaibo)

1. **Backend:** Copiar endpoints de tienda-caracas en server.js y cambiar a tienda-maracaibo
2. **Base de Datos:** Crear tabla tienda_maracaibo con misma estructura
3. **Frontend:** Crear tienda-maracaibo-spa.js copiando tienda-caracas-spa.js y cambiar:
 - API_BASE_URL → /api/tienda-maracaibo
 - Variables: allDataMB (o similar)
 - Funciones: initTiendaMaracaibo(), etc.
4. **HTML:** Agregar sección contentPagos en panel.html (reemplazar placeholder)
5. **CSS:** Agregar estilos en panel.css o crear tienda-maracaibo.css
6. **Navegación:** Agregar nav-maracaibo en sidebar (ya existe pero apunta a placeholder)
7. **Roles:** Verificar que operadores de Maracaibo vean solo esta tienda

12.2 Para Agregar un Nuevo Campo a Clientes

1. **Base de Datos:** Agregar columna a tienda_caracas y tienda_maracay
2. **Backend (server.js):** Agregar campo a allowedFields en POST y PUT
3. **Frontend (SPA):** Agregar input en createModalElement() y fillFormData()
4. **Frontend (JS):** Agregar campo en el objeto data de guardarCambios()
5. **Tabla:** Agregar columna en createRowHTML()
6. **Filtros:** Agregar campo de filtro si aplica

12.3 Para Modificar el Cálculo de Deuda

1. **Backend:** Modificar lógica en server.js (endpoints PUT de tiendas)
 2. **Frontend:** Modificar processItemData() en ambos SPA
 3. **Verificar:** Que monto_depositados y deuda se recalculen correctamente
 4. **Estados:** Revisar getEstado() para que refleje nueva lógica
-

13. CONTACTO Y REFERENCIAS

- **Sistema:** Créditos IPSFA C.A.
- **Stack:** Node.js + Express + PostgreSQL + Vanilla JS
- **Base de datos:** PostgreSQL (nombre: credits)

- **Puerto backend:** 3000 (configurable por PORT en .env)
- **Autenticación:** JWT en localStorage (clave: token)
- **Datos de usuario:** localStorage (claves: usuario, tienda_usuario)

FIN DEL INFORME TÉCNICO Documento generado el: 2026-07-14 **Total de archivos analizados:** 16 **Total de líneas de código:** ~15,000+ **Total de funciones:** ~300+ **Total de endpoints API:** 28